



División de Ingenierías  
Dept. de Ingeniería Electrónica y Telecomunicaciones  
Informática II. Docente: Aníbal José Guerra Soler

## Desafío 1: Sweet Crush

Estudiantes : Andrea Julieth Arias Cantillo, Santiago García Naranjo  
{andrea.j.arias, sgarcia.naranjo}@udea.edu.co  
20 de septiembre de 2026

### 1. Introducción

El presente informe recoge el desarrollo del proyecto Sweet Crush, realizado para la asignatura Informática 2 de la Universidad de Antioquia.

El proyecto consiste en la implementación de un tablero de juego en el que cada ficha ocupa 3 bits de memoria. A partir de esta representación se desarrollaron las principales operaciones del juego, como la generación y eliminación de fichas, la detección de combinaciones, las cascadas y la inserción o eliminación de filas y columnas.

Para organizar el desarrollo, el programa se dividió en varios módulos según la función que cumple cada uno. De esta manera, el manejo de la memoria y las operaciones sobre el tablero se mantienen separados de la lógica del juego, las modificaciones de sus dimensiones, las validaciones y la interacción con el usuario.

La implementación se realizó en C++, utilizando archivos de cabecera y de código fuente independientes.

### 2. Análisis del problema y consideraciones para la solución

Desde el inicio, uno de los principales retos del proyecto fue encontrar una forma de representar el tablero utilizando la menor cantidad de memoria posible. La condición de trabajar con 3 bits por ficha llevó a utilizar un arreglo dinámico de `unsigned char`, sobre el cual se realizan las operaciones necesarias para leer, escribir y eliminar cada posición.

Con 3 bits se pueden representar hasta ocho estados:

$$2^3 = 8$$

En el programa se utilizan los valores de 1 a 6 para las fichas y el valor 0 para representar una posición vacía. Debido a que las fichas no necesariamente coinciden con los límites de un byte, algunas pueden quedar repartidas entre dos bytes. Esto se tiene en cuenta mediante desplazamientos y máscaras al momento de acceder a ellas.

A partir de las dimensiones ingresadas por el usuario se calcula el espacio necesario para almacenar el tablero. Como cada ficha ocupa 3 bits, la cantidad total de bits requerida es:

$$\text{bits necesarios} = \text{filas} \times \text{columnas} \times 3$$

Luego se obtiene la cantidad de bytes que se deben reservar:

$$\text{bytes necesarios} = \left\lceil \frac{\text{filas} \times \text{columnas} \times 3}{8} \right\rceil$$

En la implementación, este redondeo se realiza mediante división entera:

$$\text{bytes necesarios} = \frac{\text{bits necesarios} + 7}{8}$$

También se calculan los bits que quedan disponibles en el último byte:

$$\text{bits extras} = \text{bytes necesarios} \times 8 - \text{bits necesarios}$$

Estos bits dependen directamente de las dimensiones seleccionadas y son tenidos en cuenta en las operaciones sobre el tablero.

Para establecer un límite de tamaño, se tomó como referencia un máximo de 65535 bytes para el tablero. A partir de este valor se obtiene la cantidad máxima de casillas que pueden representarse con 3 bits por ficha:

$$\text{máximo de casillas} = \frac{65535 \times 8}{3} = 174760$$

Una vez conocidas las filas, el límite de columnas se obtiene a partir de esta cantidad:

$$\text{máximo de columnas por memoria} = \frac{\text{máximo de casillas}}{\text{filas}}$$

Además del límite de memoria, se tuvo en cuenta el espacio disponible para mostrar el tablero en consola. Se estableció un ancho de 80 caracteres, de los cuales 4 se reservan para el margen de las filas y aproximadamente 3 por cada columna:

$$\text{máximo de columnas por pantalla} = \frac{80 - 4}{3} = 25$$

Por lo tanto, el límite final de columnas corresponde al menor valor entre la capacidad de memoria y el espacio disponible para la visualización:

$$\text{máximo de columnas} = \min \left( 25, \frac{\text{máximo de casillas}}{\text{filas}} \right)$$

Las dimensiones se validan antes de reservar la memoria, de manera que el tablero generado se mantenga dentro de los límites establecidos.

Durante el desarrollo también se tuvo que resolver el manejo de las combinaciones. Una misma ficha puede pertenecer a una combinación horizontal y a una vertical, por lo que no se elimina inmediatamente durante el recorrido. En su lugar, se utiliza un mapa auxiliar para marcar las posiciones que deben eliminarse y procesarlas posteriormente.

Después de una combinación pueden quedar espacios vacíos dentro del tablero. La subrutina de cascada reorganiza las fichas para ocupar estos espacios y genera nuevas fichas en las posiciones que continúan vacías. Si las nuevas fichas producen otra combinación, el proceso se repite.

Las operaciones que puede realizar el usuario durante una partida son la eliminación de fichas, filas y columnas, la adición de filas y columnas y la finalización del juego. Cada operación cuenta con su propia función y modifica el tablero conservando la representación utilizada en memoria.

Finalmente, el programa se organizó en módulos según las responsabilidades que fueron apareciendo durante el desarrollo:

- **tablero:** Concentra el manejo de la memoria y las operaciones sobre los bits.
- **juego:** Contiene las combinaciones y las cascadas.
- **modificaciones:** Se encarga de insertar y eliminar filas y columnas.
- **validaciones:** Controla las entradas del usuario.
- **interfaz:** Maneja la interacción y presentación de la información.
- **main:** Coordina el funcionamiento general.

### 3. Esquema de tareas para el desarrollo de los algoritmos

El desarrollo del programa se realizó por etapas, separando las tareas de acuerdo con las funciones que debía cumplir cada parte de la solución. Esto permitió ir probando el funcionamiento del tablero a medida que se incorporaban nuevas operaciones y mantener una organización clara entre los módulos.

Las principales tareas fueron:

1. **Definir la representación del tablero:** establecer el uso de 3 bits por ficha y la representación del tablero mediante memoria dinámica.
2. **Calcular las dimensiones y la memoria:** determinar los bits, bytes y bits extras necesarios a partir de las dimensiones del tablero, junto con los límites de memoria y visualización.
3. **Implementar el acceso a las fichas:** desarrollar las funciones para obtener, escribir y eliminar fichas, considerando las posiciones que pueden quedar distribuidas entre dos bytes.
4. **Generar y llenar el tablero:** implementar la generación aleatoria de fichas y el llenado inicial del tablero.
5. **Detectar combinaciones:** recorrer el tablero en sentido horizontal y vertical para identificar grupos de tres o más fichas iguales y marcar las posiciones correspondientes.
6. **Implementar las cascadas:** reorganizar las fichas después de una eliminación, completar los espacios vacíos y generar nuevas fichas cuando sea necesario.
7. **Implementar las modificaciones del tablero:** desarrollar las operaciones para insertar y eliminar filas y columnas, actualizando la memoria y las dimensiones del tablero.
8. **Desarrollar la interfaz:** incorporar el menú, la visualización del tablero, el monitor de memoria, las estadísticas de las jugadas y el resumen final.
9. **Separar las validaciones:** crear el módulo **validaciones** para centralizar la lectura y comprobación de las entradas numéricas y sus límites.
10. **Integrar y probar:** conectar los diferentes módulos desde **main** y realizar pruebas sobre las operaciones del tablero, verificando tanto su funcionamiento como el manejo de la memoria.

Esta división permitió mantener separadas las responsabilidades del programa. **tablero** se encarga de la memoria y las operaciones sobre los bits, **juego** de las combinaciones y cascadas, **modificaciones** de los cambios en las dimensiones, **validaciones** de las entradas, **interfaz** de la interacción con el usuario y **main** de coordinar el funcionamiento general.

## 4. Algoritmos implementados

### 4.1. Cálculo de dimensiones y memoria

Antes de crear el tablero, el programa determina cuántos bits y bytes son necesarios para almacenar todas sus fichas. Como cada posición utiliza 3 bits, el número de bits depende directamente de las dimensiones seleccionadas.

El cálculo utilizado conceptualmente es:

$$\text{bits necesarios} = \text{filas} \times \text{columnas} \times 3$$

Posteriormente se calcula la cantidad de bytes mediante:

$$\text{bytes necesarios} = \frac{\text{bits necesarios} + 7}{8}$$

utilizando división entera.

También se calculan los bits que quedan libres dentro del último byte:

```
1 bitsExtras = bytes * 8 - totalBits;
```

Estos valores son utilizados posteriormente por las funciones que acceden a las fichas.

### 4.2. Representación de las fichas

Cada ficha utiliza 3 bits. Esto permite representar ocho valores diferentes:

$$2^3 = 8$$

El programa utiliza principalmente los valores de 1 a 6 para las fichas y el valor 0 para representar una posición vacía.

Los bits válidos de las fichas se organizan comenzando desde el bit menos significativo. Cuando el número total de bits necesarios no ocupa completamente el primer byte de la trama, los bits que no se utilizan se agrupan en la parte izquierda, es decir, en los bits más significativos. Estos bits corresponden a los `bitsExtras` calculados para el tablero.

Esta distribución se tiene en cuenta al calcular la posición de cada ficha. En las funciones de acceso se utiliza:

```
1 bitInicio = bitsExtras + (fil * totalCols + col) * 3;
```

De esta forma, `bitsExtras` determina la posición inicial de la trama válida y cada ficha posterior ocupa consecutivamente 3 bits. Esto permite mantener los bits no utilizados agrupados en la parte izquierda y las fichas válidas desde la zona del bit menos significativo.

La información se almacena en un arreglo:

```
1 unsigned char* ptrTablero;
```

La memoria se reserva dinámicamente mediante:

```
1 new unsigned char[bytesTab]
```

y se libera mediante:

```
1 delete[] ptrTablero;
```

Esto permite que el tamaño del tablero pueda modificarse durante la ejecución.

### 4.3. Lectura y escritura mediante bits

Para acceder a una ficha se calcula la posición de sus tres bits dentro del arreglo. Dependiendo de la posición, los tres bits pueden encontrarse dentro del mismo byte o quedar divididos entre dos bytes.

La función `obtenerFicha` contempla estos casos utilizando desplazamientos y máscaras. De forma similar, `escribirFicha` modifica únicamente los bits correspondientes a la ficha sin alterar las demás posiciones del tablero.

Cuando los bits de una ficha se encuentran dentro de un mismo byte, se utiliza una máscara para extraerlos:

```
1 unsigned char mascara = 7 << posicionBit;
```

Cuando la ficha queda dividida entre dos bytes, se procesan por separado los bits correspondientes a cada uno de ellos. Esta consideración es necesaria por la forma compacta en que se almacena el tablero.

La eliminación de una ficha se realiza escribiendo el valor 0 en su posición:

```
1 eliminarFicha(  
2 ptrTablero ,  
3 fila ,  
4 column ,  
5 totalCols ,  
6 bitsExtras  
7 );
```

### 4.4. Generación aleatoria

Las fichas iniciales y las nuevas fichas generadas durante el juego se obtienen mediante una función aleatoria.

La función utilizada genera valores entre 1 y 6:

```
1 unsigned char generarFichaRandom()  
2 {  
3 return (rand() % 6) + 1;  
4 }
```

La semilla del generador se inicializa al comenzar el programa utilizando la hora actual.

### 4.5. Llenado inicial del tablero

Después de reservar la memoria, el tablero se llena recorriendo sus posiciones y escribiendo una ficha aleatoria en cada una.

El llenado utiliza las mismas funciones de acceso bitwise utilizadas durante el resto del programa. De esta manera, la generación inicial mantiene la misma representación compacta que se utiliza durante la partida.

### 4.6. Detección de combinaciones

El algoritmo de combinaciones realiza dos búsquedas.

La primera recorre cada fila para identificar secuencias horizontales de tres o más fichas iguales. La segunda recorre cada columna para identificar secuencias verticales de tres o más fichas iguales.

Para evitar eliminar una misma ficha más de una vez, se crea un mapa auxiliar dinámico:

```
1 bool* mapaAuxiliar = new bool[totalCasillas]();
```

Cuando una ficha pertenece a una combinación, su posición se marca en este mapa. La eliminación se realiza después de terminar las búsquedas, por lo que una ficha que pertenezca simultáneamente a una combinación horizontal y una vertical solo se procesa una vez.

La búsqueda horizontal identifica una secuencia cuando encuentra tres o más fichas consecutivas con el mismo valor. El mismo procedimiento se aplica posteriormente en sentido vertical.

Una vez terminada la búsqueda, las posiciones marcadas se recorren y se eliminan del tablero.

#### 4.7. Puntuación y estadísticas

Por cada ficha destruida se agregan 10 puntos.

La puntuación se actualiza mediante la cantidad de fichas eliminadas en cada ronda:

$$\text{puntuación} = \text{fichas destruidas} \times 10$$

El programa también mantiene diferentes estadísticas:

- puntuación acumulada;
- fichas eliminadas;
- rondas de combinación;
- combos detectados;
- intervenciones realizadas directamente por el usuario.

Estas variables permiten mostrar tanto el resultado de cada jugada como el resumen final de la partida.

#### 4.8. Algoritmo de cascada

Después de eliminar las fichas que forman combinaciones, pueden quedar espacios vacíos dentro del tablero.

La función `subrutinaCascada` recorre las columnas desde la parte inferior hacia arriba. Cuando encuentra una posición vacía, busca una ficha no vacía en las posiciones superiores y la desplaza hacia abajo.

Después de desplazar las fichas, las posiciones que continúan vacías en la parte superior se llenan con nuevas fichas aleatorias.

Una vez realizada la cascada, el programa vuelve a comprobar el tablero para determinar si las nuevas posiciones producen otra combinación.

#### 4.9. Proceso completo de una combinación

La detección de combinaciones y las cascadas trabajan de manera conjunta. Cuando una jugada modifica el tablero, se ejecuta la cascada y posteriormente se verifica nuevamente la existencia de combinaciones.

El proceso se repite mientras se encuentren nuevas combinaciones:

```

1 do {
2
3   '''
4   subrutinaCascada(
5       pTab,
6       filas,
7       columnas,
8       bitsExtras
9   );
10
11   huboCombinacion = subrutinaCombinaciones(
12       pTab,
13       filas,
14       columnas,
15       bitsExtras,
16       puntuacion,
17       totalFichasElim,
18       combDetectadas,
19       combosDetectados
20   );
21   '''
22
23 } while (huboCombinacion);

```

De esta manera se pueden producir varias rondas de combinaciones a partir de una misma jugada.

#### 4.10. Eliminación de filas

Cuando el usuario elimina una fila, se calcula el nuevo tamaño del tablero. Se crea un nuevo bloque de memoria y se copian todas las filas excepto la que fue seleccionada. El procedimiento se apoya en la función `copiarTableroExcluyendo`, que permite indicar la fila o columna que no debe copiarse.

Después se libera el bloque anterior y se actualizan:

- el puntero del tablero;
- el número de filas;
- los bits extras;
- los bytes reservados.

Para las eliminaciones se utiliza una condición del 65 % para determinar si es necesario reducir la memoria reservada. Si el espacio utilizado después de la eliminación representa menos del 65 % de la memoria actualmente reservada, se crea un nuevo bloque con el tamaño requerido, se copia la información del tablero y se libera el bloque anterior.

#### 4.11. Eliminación de columnas

El proceso para eliminar una columna es similar al utilizado para eliminar una fila.

Se crea un nuevo tablero, se copian las columnas que deben conservarse y se omite la columna seleccionada.

Finalmente se libera la memoria anterior y se actualizan las dimensiones y los datos relacionados con la memoria.

#### 4.12. Inserción de filas

Para insertar una fila se crea un nuevo tablero con las dimensiones actualizadas.

Las filas existentes se copian teniendo en cuenta la posición seleccionada por el usuario. La nueva fila se llena con fichas generadas aleatoriamente.

Una parte importante del algoritmo consiste en desplazar las filas que se encuentran después de la posición de inserción para conservar su orden.

Después se libera el bloque anterior y se actualiza la información del tablero.

#### 4.13. Inserción de columnas

La inserción de una columna sigue un procedimiento equivalente.

Las columnas anteriores a la posición seleccionada permanecen en su lugar, la nueva columna se llena con fichas aleatorias y las columnas posteriores se desplazan una posición.

Finalmente se reemplaza el bloque anterior de memoria por el nuevo bloque.

#### 4.14. Validación de dimensiones y entradas

El programa establece límites para evitar que se solicite un tablero que supere la memoria establecida o que no pueda visualizarse correctamente en la consola.

El número máximo de casillas está relacionado con el límite de 65535 bytes y los 3 bits utilizados por cada ficha:

$$\text{máximo de casillas} = \frac{65535 \times 8}{3}$$

Una vez conocidas las filas, el máximo de columnas permitido por memoria se calcula como:

$$\text{máximo de columnas por memoria} = \frac{\text{máximo de casillas}}{\text{filas}}$$

Además, la consola establece un límite de 25 columnas para la visualización. Por esta razón se utiliza el menor de los dos límites:

$$\text{máximo de columnas} = \min \left( 25, \frac{\text{máximo de casillas}}{\text{filas}} \right)$$

La validación de las entradas se centralizó posteriormente en los archivos `validaciones.h` y `validaciones.cpp`. El módulo proporciona la función `leerOpcionSegura`, que recibe un límite mínimo, un límite máximo y un mensaje.

La entrada se recibe mediante un arreglo de caracteres y se comprueba que todos sus caracteres correspondan a dígitos antes de convertirla a un valor numérico. También se verifica que el valor se encuentre dentro del intervalo permitido. Si la entrada no cumple alguna de estas condiciones, se muestra un mensaje de error y se solicita nuevamente.

La función es utilizada tanto desde `main.cpp` como desde `interfaz.cpp`, evitando duplicar la lógica de validación en ambos módulos.

#### 4.15. Flujo principal del programa

Finalmente, `main.cpp` coordina los diferentes algoritmos. Primero inicializa el generador aleatorio, muestra la bienvenida y solicita las dimensiones utilizando el módulo de validaciones. Con estos datos se calcula la memoria necesaria, se crea el tablero y se realiza su llenado inicial.

Posteriormente se procesan las combinaciones iniciales y se entra en el ciclo principal del menú. Cada opción seleccionada por el usuario es enviada a `ejecutarOpcion`, donde se realiza la operación correspondiente y, cuando es necesario, se vuelven a ejecutar las combinaciones y las cascadas.

Al finalizar la ejecución se libera la memoria dinámica del tablero y se muestra el resumen de las estadísticas obtenidas durante la partida.

### 5. Organización modular

El programa se divide en diferentes módulos, cada uno encargado de un conjunto específico de operaciones. Esta organización permite separar el manejo de la memoria, la lógica del juego, las modificaciones del tablero, las validaciones y la interacción con el usuario.

#### 5.1. `tablero.h` y `tablero.cpp`

Este módulo contiene las operaciones relacionadas con la representación del tablero en memoria. Sus principales responsabilidades son:

- reservar y liberar la memoria del tablero;
- leer fichas;
- escribir fichas;
- eliminar fichas;
- generar fichas aleatorias;
- llenar el tablero;
- mostrar el tablero en bits;
- mostrar el tablero mediante símbolos.

Las operaciones sobre las fichas se realizan directamente sobre el arreglo dinámico mediante operaciones a nivel de bits.

#### 5.2. `juego.h` y `juego.cpp`

Este módulo contiene la lógica relacionada con las combinaciones y las cascadas. Sus principales responsabilidades son:

- detectar combinaciones horizontales;
- detectar combinaciones verticales;
- marcar las fichas que deben eliminarse;
- calcular la puntuación;

- actualizar las estadísticas;
- ejecutar las cascadas;
- generar nuevas fichas después de las eliminaciones.

Este módulo trabaja sobre el tablero recibido mediante punteros, sin encargarse directamente de la interacción con el usuario.

### 5.3. `modificaciones.h` y `modificaciones.cpp`

Este módulo se encarga de modificar las dimensiones del tablero y gestionar los cambios de memoria asociados.

Contiene las funciones para:

- calcular bytes y bits extras;
- copiar el contenido del tablero;
- eliminar filas;
- eliminar columnas;
- insertar filas;
- insertar columnas.

Cuando una operación modifica las dimensiones, se crea el nuevo bloque de memoria, se copia la información necesaria y se libera el bloque anterior.

### 5.4. `validaciones.h` y `validaciones.cpp`

Este módulo se encarga de validar las entradas numéricas realizadas por el usuario.

La función `leerOpcionSegura` comprueba que la entrada corresponda a un número entero y que se encuentre dentro del intervalo permitido.

El módulo recibe los límites mínimo y máximo como parámetros, lo que permite utilizar la misma función para diferentes entradas del programa, como las dimensiones del tablero y las opciones del menú.

La separación de esta función permitió evitar la duplicación del código de validación y mantener independiente esta lógica de la presentación de la interfaz.

### 5.5. `interfaz.h` y `interfaz.cpp`

Este módulo concentra la interacción y presentación de información al usuario.

Se encarga de mostrar:

- bienvenida;
- menú;
- monitor de memoria;
- estadísticas de cada jugada;

- resumen final.

También contiene `ejecutarOpcion`, función encargada de recibir la opción seleccionada y coordinar la ejecución de las operaciones correspondientes.

Las validaciones de entrada no se implementan directamente en este módulo, sino que se realizan mediante la función definida en `validaciones`.

## 5.6. `main.cpp`

El archivo principal coordina el funcionamiento general del programa.

Entre sus responsabilidades están:

- inicializar el generador aleatorio;
- solicitar las dimensiones mediante el módulo de validaciones;
- calcular la memoria necesaria;
- crear y llenar el tablero;
- procesar las combinaciones iniciales;
- ejecutar el menú;
- coordinar las operaciones entre los diferentes módulos;
- liberar la memoria al finalizar.

## 6. Problemas encontrados durante el desarrollo

Durante el desarrollo del proyecto se presentaron diferentes dificultades que requirieron ajustes en la implementación y en la organización del programa. Una de las principales estuvo relacionada con la representación de las fichas en memoria. Al utilizar 3 bits por ficha, algunas pueden quedar divididas entre dos bytes, por lo que fue necesario considerar diferentes casos para leer y escribir correctamente la información.

También se presentaron dificultades al modificar las dimensiones del tablero. Al insertar o eliminar filas y columnas fue necesario reorganizar la memoria, conservar las fichas correspondientes y actualizar los datos asociados al nuevo tamaño. El manejo de las combinaciones presentó otro inconveniente, ya que una misma ficha puede pertenecer simultáneamente a una combinación horizontal y una vertical. Para evitar eliminarla más de una vez se implementó un mapa auxiliar que marca las posiciones antes de realizar la eliminación.

La organización modular también requirió ajustes debido al trabajo entre los dos integrantes. A medida que aumentaron los archivos, fue necesario definir con mayor claridad la responsabilidad de cada módulo para facilitar las modificaciones y evitar confusiones.

Finalmente, la validación de entradas inicialmente se encontraba distribuida en diferentes partes del programa. Durante la organización final se creó el módulo `validaciones`, compuesto por `validaciones.h` y `validaciones.cpp`, donde se concentró la función `leerOpcionSegura`. Esto permitió reutilizar la misma lógica y separar la validación de la interacción con el usuario.

## 7. Evolución de la solución

El desarrollo del programa se realizó de manera progresiva, incorporando las funcionalidades y reorganizando el código a medida que aumentaba la complejidad de la solución.

Inicialmente se desarrolló la representación del tablero en memoria, junto con las operaciones para leer, escribir, eliminar y mostrar las fichas utilizando 3 bits. Posteriormente se incorporaron la generación aleatoria y el llenado del tablero.

Luego se implementaron las cascadas y la detección de combinaciones, incluyendo el manejo de patrones horizontales, verticales y combinaciones que pueden involucrar varias direcciones. En esta etapa también se integró el cálculo de la puntuación y las estadísticas del juego.

Después se reorganizó el programa en módulos. Primero se separó la interfaz y el programa principal, y posteriormente se creó el módulo **modificaciones**, encargado de las operaciones de inserción y eliminación de filas y columnas. Esto implicó ajustar el manejo de la memoria y mantener actualizada la información del tablero después de cada operación.

Con el crecimiento del programa también se incorporaron límites para las dimensiones del tablero, considerando la memoria disponible y el espacio de visualización en consola. A partir de esto se agregaron validaciones para controlar las entradas del usuario.

Finalmente, las validaciones que inicialmente estaban integradas en otras partes del programa se reorganizaron en los archivos **validaciones.h** y **validaciones.cpp**. Además de corregir su funcionamiento, se ajustó **main.cpp** para utilizar este nuevo módulo y evitar la duplicación de la lógica de validación.

Como resultado, la solución pasó de una implementación inicial concentrada en el tablero a una estructura modular en la que se separan el manejo de memoria, la lógica del juego, las modificaciones del tablero, las validaciones y la interacción con el usuario.

## 8. Consideraciones sobre memoria y tipos de datos

La elección de los tipos de datos se realizó teniendo en cuenta la función, el rango de valores y la cantidad de memoria que requiere cada variable. En los casos relacionados con el tablero también se consideró la necesidad de trabajar directamente con bits y bytes.

Para almacenar el tablero se utiliza:

```
1 unsigned char*
```

Se eligió este tipo porque permite acceder directamente a la memoria organizada en bytes, que es la unidad utilizada para almacenar las fichas de 3 bits. El puntero permite además reservar dinámicamente la cantidad de memoria necesaria para cada tablero.

Las dimensiones del tablero y los valores relacionados con la cantidad de bytes y bits utilizados emplean principalmente:

```
1 unsigned short
```

Se utiliza porque estos valores son siempre positivos y se encuentran dentro del rango establecido para las dimensiones y el manejo de la memoria del tablero. Al utilizar un tipo sin signo no se reserva espacio para valores negativos que no tienen sentido en estas variables.

Las estadísticas, como la puntuación y la cantidad de fichas eliminadas, utilizan:

```
1 unsigned int
```

En este caso se utiliza un tipo con mayor rango, ya que estos valores pueden aumentar durante toda la partida y no tienen valores negativos. Esto permite acumular la puntuación y las cantidades de eliminaciones sin limitarse al rango utilizado para las dimensiones del tablero.

Las fichas se manejan mediante:

```
1 unsigned char
```

Aunque cada ficha ocupa solamente 3 bits dentro del tablero, se utiliza `unsigned char` para almacenar temporalmente su valor al leerla o escribirla. Este tipo resulta adecuado porque una ficha solo puede tomar los valores utilizados por el juego y no necesita valores negativos. La representación compacta de 3 bits se mantiene al momento de almacenarla en el tablero mediante las operaciones a nivel de bits.

La memoria del tablero se administra dinámicamente mediante `new[]` y `delete[]`. Esta elección permite reservar únicamente el espacio necesario según las dimensiones seleccionadas y volver a reservarlo cuando se insertan o eliminan filas y columnas.

Además, el algoritmo de detección de combinaciones utiliza temporalmente un arreglo dinámico de tipo `bool` para marcar las posiciones que deben ser eliminadas. Se utiliza este tipo porque cada posición solamente necesita indicar dos estados: pertenece o no pertenece a una combinación. El arreglo se libera una vez finalizado el procesamiento de la combinación.

## 9. Flujo general del programa

El funcionamiento general del programa sigue una secuencia en la que primero se configura el tablero y posteriormente se ejecutan las operaciones de la partida. El flujo principal es el siguiente:

1. Inicializar el generador de números aleatorios.
2. Mostrar la bienvenida y solicitar las dimensiones del tablero, validando las entradas y los límites establecidos.
3. Calcular los bits, bytes y bits extras necesarios para representar el tablero.
4. Reservar dinámicamente la memoria y llenar el tablero con fichas aleatorias.
5. Detectar y procesar las combinaciones iniciales.
6. Mostrar el tablero y las estadísticas correspondientes.
7. Mostrar el menú y solicitar una opción al usuario mediante el módulo de validaciones.
8. Ejecutar la operación seleccionada, ya sea una eliminación, inserción, modificación o finalización de la partida.
9. Cuando la operación modifica el tablero, ejecutar las cascadas y comprobar nuevamente la existencia de combinaciones.
10. Actualizar la puntuación y las estadísticas de la partida.
11. Repetir el ciclo de interacción hasta que el usuario seleccione la opción de finalizar.
12. Liberar la memoria dinámica utilizada por el tablero.
13. Mostrar el resumen final de la partida.

De esta forma, `main.cpp` coordina el flujo general mientras las operaciones específicas son ejecutadas por los módulos correspondientes.

## 10. Resultados de la implementación

La implementación final permite trabajar con un tablero dinámico cuya representación se mantiene en memoria mediante 3 bits por ficha. Sobre esta estructura se pueden realizar las diferentes operaciones definidas para el proyecto.

El usuario puede:

- seleccionar las dimensiones iniciales del tablero;
- visualizar el tablero en bits;
- visualizar las fichas mediante símbolos;
- eliminar fichas individuales;
- eliminar filas y columnas;
- insertar filas y columnas;
- detectar y procesar combinaciones;
- ejecutar cascadas y generar nuevas fichas;
- consultar las estadísticas de cada jugada;
- consultar información sobre la memoria utilizada;
- finalizar la partida y visualizar el resumen final.

Las operaciones de inserción y eliminación actualizan dinámicamente el tamaño del tablero y la memoria asociada. De igual forma, las combinaciones detectadas se procesan mediante eliminaciones y cascadas, permitiendo que el tablero continúe modificándose durante la partida.

Las entradas del usuario son verificadas antes de ejecutar las operaciones. Los límites establecidos para las dimensiones, la memoria y la visualización permiten controlar el tamaño del tablero y evitar valores fuera del rango definido.

La organización modular también permitió mantener separadas las funciones principales del programa, facilitando la integración y las pruebas de cada parte de la solución.

## 11. Conclusiones

El desarrollo de Sweet Crush permitió aplicar diferentes conceptos de programación en C++, principalmente el manejo de punteros, arreglos, memoria dinámica, operaciones a nivel de bits y modularización. La representación de las fichas mediante 3 bits permite reducir el espacio necesario para almacenar el tablero y, al mismo tiempo, requiere implementar funciones específicas para acceder a posiciones que pueden encontrarse distribuidas entre dos bytes.

La organización del programa en los módulos **tablero**, **juego**, **modificaciones**, **validaciones** e **interfaz** permitió separar las responsabilidades de la solución y facilitar la incorporación de nuevas funcionalidades durante el desarrollo. El manejo dinámico de la memoria fue especialmente importante en las operaciones de inserción y eliminación de filas y columnas, ya que estas modifican las dimensiones y requieren reorganizar la información almacenada.

Finalmente, la integración de la detección de combinaciones, las cascadas, las modificaciones del tablero y las validaciones permitió obtener una implementación funcional en la que los diferentes módulos trabajan sobre una misma representación del tablero y mantienen actualizada la información necesaria durante la partida.